COMPILER CONSTRUCTION LAB · WS 2026/27 · PA1

A ChocoPy front-end: lexing and parsing

JFlex scanner · CUP LALR(1) parser · JSON AST with error recovery

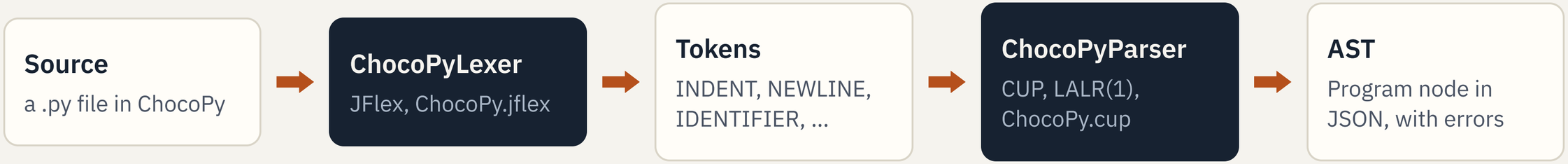
Group 4

Omar Kamel · Malak Chaouch

University of Passau
Chair of Software Engineering I

OVERVIEW

From ChocoPy source to a JSON syntax tree



33 / 33

sample tests pass, plus our own good.py and bad.py

11,329

programs parsed exactly like the reference parser

0

conflicts left in the grammar after precedences

The lexer works in two states

YYINITIAL

Start of a physical line: skip blank and comment-only lines, measure the indentation, emit INDENT or DEDENT.

first real character of the line

line break, emits NEWLINE

LINE_BODY

Inside a logical line: keywords, identifiers, literals and operators; spaces and comments are skipped.

Maximal munch, then rule order

classic is one IDENTIFIER, **class** is a keyword: keyword rules come before the identifier rule.

Reserved Python keywords

break, **import**, **lambda** and 13 more become RESERVED: never an identifier, never in a grammar rule.

The lexer never fails

Unknown input such as **$** or **&** becomes UNRECOGNIZED, so the parser reports it with its position.

Indentation: one rule, a stack and yypushback

```
{WhiteSpace}* [^ \t\r\n#] {
  yypushback(1);          // keep the first char
  int indentation = indentationOf(yytext());
  if (indentation > indentStack.peek()) {
    indentStack.push(indentation);
    yybegin(LINE_BODY);
    return INDENT;
  }
  if (indentation < indentStack.peek()) {
    indentStack.pop();
    yypushback(width);    // fire again
    return DEDENT;
  }
  yybegin(LINE_BODY);     // same level
}
```

Deeper

push the level, one INDENT

Same level

no token, continue the line

Shallower

pop, one DEDENT, push the whitespace back: the rule fires again until the level matches

Tabs advance to the next multiple of 8. At the end of input: NEWLINE if missing, one DEDENT per open level, EOF. **ChocoPy.jflex**, lines 162–192 and 283–298

Example: the tokens of two nested blocks

Line	Source	Stack	Tokens
1	if x:	[0]	IF IDENTIFIER COLON NEWLINE
2	····if y:	[0, 4]	**INDENT** IF IDENTIFIER COLON NEWLINE
3	········a = 1	[0, 4, 8]	**INDENT** IDENTIFIER EQ INTEGER NEWLINE
4	(blank line)	[0, 4, 8]	none
5	····# done	[0, 4, 8]	none: comment-only lines are skipped
6	b = 2	[0]	**DEDENT DEDENT** IDENTIFIER EQ INTEGER NEWLINE
end		[0]	EOF

Line 6 runs the indentation rule three times: two DEDENTs, then the same level. The comment on line 5 is indented by 4, but it neither closes a block nor breaks it.

LEXER · OMAR

Literals and the edge cases we checked

Input	Token	Why
"abc"	IDSTRING	looks like an identifier, so it may name a type
"a b"	STRING	any other printable ASCII text
"He\"llo"	STRING He"llo	escapes \" \n \t \\ are applied to the value
"Hell\o"	UNRECOGNIZED	illegal escape: the whole literal is one bad token
2147483648	UNRECOGNIZED	larger than 2^31 – 1
007	INTEGER ×3	0, 0, 7: no leading zeros, maximal munch
\r, \n, \r\n	NEWLINE	all three line ends, on every platform

Every row was compared with the reference lexer. Fuzzing found one crash here: a 20-digit literal overflowed **long**; now any number that does not fit an **int** is UNRECOGNIZED.

Expressions: one rule per precedence level

```
/* lowest precedence first */
expression  ::= or_expression
              | or_expression IF … ELSE …
or_expression ::= or_expression OR and_expression
and_expression ::= … AND not_expression
not_expression ::= NOT not_expression
               | comparison
comparison  ::= arithmetic LT arithmetic | …
arithmetic  ::= arithmetic PLUS term | …
term        ::= term TIMES factor | …
factor      ::= MINUS factor | atom
postfix     ::= postfix DOT IDENTIFIER | …
```

The levels encode precedence and associativity: CUP finds 0 conflicts, even without precedence declarations.

> **`a < b < c`**
>
> error at the second <: both operands of a comparison are arithmetic

> **`True == not False`**
>
> error at not: not sits above comparison, not below it

> **`-a.b[0]`**
>
> is −(a.b[0]): postfix is the last level, so it binds tightest

PARSER · MALAK

Building the AST: elif and exact locations

```
if a:
    x = 1
elif b:
    x = 2
else:
    x = 3
```

IfStmt condition a, thenBody [x = 1], elseBody:

> **IfStmt** condition b, thenBody [x = 2], elseBody [x = 3]
>
> starts at the elif keyword, ends at the DEDENT of the else block

Leftmost to rightmost

A node spans its CUP symbols: (1 + 2) * 3 covers columns 1–11, parentheses included.

Statements stop early

A statement node ends before its NEWLINE, but a function ends where that NEWLINE ends.

Blocks end at DEDENT

if, while and for end at the DEDENT, which spans the indentation of the next line.

ERROR RECOVERY · MALAK

Error recovery with the error symbol

1 · STATEMENTS

A broken line

Without an error rule, CUP stops at the first error and checks nothing after it.

Fix: skip to the end of the line and continue with the next statement.

```
statement ::=
  simple_statement NEWLINE
| compound_statement
| error NEWLINE
```

2 · FUNCTIONS

Broken headers

After an error in def f(a, b) -> int: the indented body is left behind.

Fix: skip the header, then still parse the body, so its errors are reported.

```
function_definition ::=
  DEF … COLON NEWLINE
    INDENT function_body DEDENT
| DEF error NEWLINE
    INDENT function_body DEDENT
```

3 · CLASSES

A bad member

One wrong line in a class body should not lose the other members.

Fix: skip the bad member or header line, keep the other members.

```
class_decl_list ::= …
| class_decl_list
    error NEWLINE
declaration ::= …
| CLASS error NEWLINE
    INDENT class_decl_list DEDENT
```

ERROR RECOVERY · MALAK

Result on our bad.py

9

errors from our parser, each one also reported by the reference

17

errors from the reference parser, with follow-ups at DEDENTs

```
print(1 + )                          ← 3:11 RPAREN
print(2)                             kept
…
value: int = 1 + 2                   ← 11:6 COLON
def broken(a: int, b) -> int:
    …                                ← 18:1 DEDENT
class Pair(object):
    …
    third: int = 2                   ← 24:10 COLON
if 1 < value < 3:                    ← 28:14 LT
value = value && 1                   ← 33:15 UNRECOGNIZED
print(True == not False)             ← 34:15 NOT
while value > 0                      ← 38:16 NEWLINE
```

TESTING · OMAR

The reference parser as a test oracle

35 sample and student tests

99 hand-written edge cases

11,195 random mutants

Both parsers in one JVM

valid program: the whole JSON must be equal, locations included

invalid program: the first error at the same line and column

11,329

identical, 0 failures

Mutants

delete, insert, duplicate or swap tokens, add or remove indentation in every test file

Bugs it found in our code

a crash on huge integers, DEDENT columns at EOF, the start of a program that begins with pass

TESTING · OMAR

What we found in the reference and the material

Where	Finding
Reference lexer	No NEWLINE at end of input without a final line break; hidden because the driver appends "\n"
Reference locations	The k-th DEDENT at EOF sits at column k of a line after the file, e.g. [1, 1, 5, 3] for a 4-line file
Reference messages	"Parse error near token DEDENT: x" shows the text of the previous token
Local --test mode	Does not compare locations, so wrong locations still pass
Specification	Kind NonlocalDecl is NonLocalDecl in the AST and outputs; typo "stater code"
Manual 2.8.6	"outputs outputs its printed form"

Each item has a small input and the expected output in our README.

SUMMARY

Who did what

Omar Kamel

LEXER AND TESTING

ChocoPy.jflex: line structure, INDENT and DEDENT, tabs, string and integer literals, reserved keywords, UNRECOGNIZED; good.py; differential testing against the reference

Malak Chaouch

GRAMMAR, AST AND RECOVERY

ChocoPy.cup: declarations, statements, expressions and precedences, elif desugaring, exact node locations, error rules; bad.py

We wrote the README together and reviewed every part of each other's code, so both of us can explain the whole project. About 30 hours in total.

FOR THE Q&A

Where each part lives in the code

Part	File	Lines
Indentation stack and the tab rule	ChocoPy.jflex	83-103
Start of a line: INDENT and DEDENT	ChocoPy.jflex	162-192
Keywords, literals, operators	ChocoPy.jflex	198-279
End of input	ChocoPy.jflex	283-298
Program, declarations and classes	ChocoPy.cup	149-200
Functions and their bodies	ChocoPy.cup	202-235
Statements, blocks and elif	ChocoPy.cup	255-309
Expressions, one rule per level	ChocoPy.cup	311-392

PA1 · GROUP 4

Thank you. Questions?

Lexer, grammar, error recovery, testing: ask any of us.

```
mvn clean package
java -cp "chocopy-ref.jar;target/assignment.jar" chocopy.ChocoPy --pass=s --dir src/test/data/pa1 --test
```